

CUSTOMER REVIEW INTELLIGENCE

NLP-Based Analysis of Amazon Customer Reviews

Ironhack NLP Project Report | Version 1

Executive Summary

This project builds an end-to-end Natural Language Processing (NLP) system for extracting useful business insights from Amazon customer reviews. The solution combines sentiment classification, unsupervised product clustering, lightweight generative summarization, and a deployed Streamlit dashboard. The goal of Version 1 was to deliver a complete working product first, then leave model and user-experience improvements for later iterations.

Project Objectives

- Classify reviews as Negative, Neutral, or Positive and compare multiple sentiment approaches.
- Group products into a small set of meaningful meta-categories using unsupervised clustering.
- Generate short category-level customer summaries using a lightweight generative model.
- Expose the results through a web application that supports exploration and live sentiment prediction.

Dataset and Final Scope

The main working dataset was cleaned from the Amazon review data provided for the project. Essential missing values were removed, duplicate product-review pairs were removed, and the data was reduced to the fields needed for modeling and application development.

Cleaned reviews	Unique products	Sentiment classes	Meta-categories	Deployment
34,624	38	3	6	Streamlit Cloud

Live application: <https://customerreviewproject-n7miuckjup3npneptcqr8.streamlit.app/>

1. Data Preparation and Sentiment Analysis

Data Preparation

The source file contained 34,660 rows and 21 columns. The working dataframe retained product name, product ID, brand, category, rating, review text, review title, and recommendation information. Reviews without the essential product ID, rating, or review text were removed. Duplicate rows with the same product ID and review text were also removed to reduce repeated evidence and potential train/test leakage.

Ratings were converted to sentiment labels using a simple business rule: ratings 1-2 = Negative, rating 3 = Neutral, and ratings 4-5 = Positive. The resulting dataset was highly imbalanced: approximately 93.3% Positive, 4.3% Neutral, and 2.3% Negative. Therefore, accuracy alone was not treated as sufficient; Macro F1, class precision/recall, and confusion matrices were also considered.

Train / Validation / Test Design

The cleaned data was split into 70% training, 15% validation, and 15% test sets using stratification so that the class distribution remained consistent across all three subsets. The test set was kept untouched until the final model selection was completed.

Sentiment Models Compared

Model	Purpose	Validation Accuracy	Macro F1
TF-IDF + Logistic Regression	Traditional baseline	0.8708	0.51
Pretrained Amazon DistilBERT	Domain-specific pretrained model	0.8922	0.60
Fine-tuned DistilBERT	Custom balanced fine-tuning	0.8279	0.53

Model Selection and Test Result

The pretrained Amazon DistilBERT model (SebasLopez-ai/distilbert-amazon-reviews-sentiment) provided the strongest validation balance and was selected for the final application. On the untouched test set it achieved approximately 89.5% accuracy and 0.63 Macro F1. Its class-level test recall was strong for Negative reviews (0.75) while retaining high Positive performance (0.92 recall).

The custom fine-tuned model improved minority-class recall compared with the logistic baseline, but its aggressive handling of the balanced training subset caused more Positive reviews to be predicted as Neutral or Negative. This demonstrated why class-specific metrics were necessary for model selection.

2. Product Clustering

Why Clustering Was Needed

The dataset contained many long, overlapping raw category strings. The project therefore used unsupervised clustering to reduce the catalog to a small number of interpretable product groups. KMeans was selected because it is lightweight, easy to explain, works well with TF-IDF vectors, and does not require labelled target categories.

Clustering Development

The first experiment combined product name, category information, and review text. This produced weak separation because generic review vocabulary such as “great”, “good”, “love”, and “easy” dominated the signal. Review text was therefore removed from the clustering representation. The next representation used product name plus category text; this was more relevant, but categories were still noisy and several product names were missing.

The final representation increased the importance of the product name by repeating it before the category text. This intentionally weighted highly descriptive product names such as “Echo”, “Kindle Voyage”, and “Fire HD” more strongly than generic category tokens.

Cluster Selection

K	Silhouette Score	Interpretation
4	0.2313	Reasonable but some product families remained mixed
5	0.2601	Better mathematical separation
6	0.2673	Best score and acceptable business interpretation

Final Meta-Categories

• Accessories & Power	• Echo & Smart Home
• Fire Tablets - Standard	• Fire HD Tablets
• Kindle Fire 16GB Tablets	• Kindle Voyage / E-readers

KMeans generated numeric cluster IDs only. Product lists and top TF-IDF terms were inspected manually, and the six business-friendly names above were assigned after interpretation. The resulting meta-category was then merged back into all 34,624 review-level rows for use in the application.

3. Generative Summarization and Application Architecture

Generative AI Summarization

The project uses google/flan-t5-small from Hugging Face as the Version 1 generative model. It was selected because it is lightweight enough for local CPU experimentation and supports instruction-style text generation. Review samples were drawn across Positive, Neutral, and Negative sentiment groups so that the summarizer could see both strengths and complaints.

During experimentation, FLAN-T5-small sometimes produced very short or generic outputs, and a dedicated T5 summarization model was also tested. For the MVP, the decision was to keep FLAN-T5-small and treat summary quality as a known limitation rather than delay deployment. Category summaries were generated offline and exported to category_summaries.json.

Why Pre-generate Summaries?

- The same six category summaries do not need to be regenerated on every page load.
- Pre-generation reduces CPU use and removes generation latency for deployed users.
- The JSON artifact is simple, portable, and easy for the Streamlit app to load.

Application Architecture

Component	Runtime behavior	Artifact / model
Category Insights	Offline results loaded instantly	app_reviews.csv + category_summaries.json
Product Explorer	Offline product assignments	products.csv
Review Analyzer	Live model inference	Amazon DistilBERT from Hugging Face

The Streamlit app therefore mixes precomputed analytics with live inference. Clustering and summarization are performed offline, while a new review entered by a user is sent to the pretrained DistilBERT model at runtime. Streamlit caching is used so that data files and the sentiment model are not repeatedly reloaded.

Deployment

The application was deployed to Streamlit Community Cloud from the GitHub repository. A minimal deployment environment was validated from a clean Python 3.12 virtual environment using only Streamlit, Pandas, Transformers, and PyTorch as direct requirements.

4. Results, Limitations and Next Steps

Final Version 1 Outcome

Version 1 successfully delivers a complete NLP workflow from raw customer-review data to a publicly accessible application. The system demonstrates three different NLP paradigms in one product: supervised classification, unsupervised clustering, and generative text summarization.

Cleaned review records	34,624
Unique products	38
Selected sentiment model	Pretrained Amazon DistilBERT
Final sentiment test accuracy	~89.5%
Final sentiment Macro F1	~0.63
Final product clusters	6 (Silhouette 0.2673)

Known Limitations

- The original review distribution is extremely Positive-heavy, so minority-class performance remains challenging.
- FLAN-T5-small is intentionally lightweight and can generate summaries that are short, generic, or partially extractive.
- TF-IDF + KMeans captures lexical similarity rather than deeper semantic product relationships.
- The catalog contains only 38 unique product IDs, and several original product names are missing.
- Category summaries are currently static pre-generated artifacts rather than live generation.

Recommended Version 2 Improvements

- Upgrade summarization using a stronger instruction model or improved hierarchical summarization pipeline.
- Replace or compare TF-IDF clustering with sentence embeddings for better semantic grouping.
- Add product recommendation/ranking, product search, and richer business metrics.
- Collect user feedback and use observed failures to prioritize iterative model improvements.
- For a more product-oriented architecture, separate a modern web frontend from a Python/FastAPI ML service.

Conclusion

The project achieved its main objective: a working customer-review intelligence application that combines multiple NLP methods and is deployed for real user interaction. The most important engineering decision was to prioritize a complete, testable Version 1 instead of endlessly optimizing individual models. The deployed system now provides a concrete baseline that can be improved iteratively using measurable model performance and user experience feedback.

Build → Deploy → Test → Learn → Improve